

SOLUTION

Q1: Water Jug Problem (4-Gallon and 3-Gallon Jugs)

1. Water Jug Problem

Solution

The Water Jug Problem is solved by performing a sequence of valid operations on the two jugs. Initially, both the **4-gallon** and **3-gallon** jugs are empty. Water is filled into the 4-gallon jug and then poured into the 3-gallon jug. By repeating the operations of filling, emptying, and pouring, the amount of water in the 4-gallon jug is adjusted until **exactly 2 gallons** remain. The process stops when the required goal state is achieved.

Step-by-Step Solution

Step	4-Gallon Jug	3-Gallon Jug	Action
Initial	0	0	Both jugs are empty
1	4	0	Fill the 4-gallon jug
2	1	3	Pour water from the 4-gallon jug into the 3-gallon jug
3	1	0	Empty the 3-gallon jug
4	0	1	Pour the remaining 1 gallon into the 3-gallon jug

Step	4-Gallon Jug	3-Gallon Jug	Action
5	4	1	Fill the 4-gallon jug again
6	2	3	Pour water into the 3-gallon jug until it becomes full

Goal Achieved: The 4-gallon jug contains exactly 2 gallons.

2. Mars Rover – Intelligent Agent

Solution

The Mars Rover is an intelligent agent that performs scientific exploration on the surface of Mars. It receives information through sensors, processes the collected data, and performs suitable actions based on the current situation. The rover captures images, collects rock and soil samples, analyzes the environment, avoids obstacles, and transmits the collected information to Earth. Since communication with Earth is delayed, the rover must make many decisions independently. A **Goal-Based Agent** is the most suitable architecture because it selects actions to achieve specific mission objectives.

Answers

Percepts

- Camera images
- Rock and soil samples

- Temperature
- Atmospheric pressure
- Obstacle information
- Battery status

Operating Environment

- Partially Observable
- Dynamic
- Sequential
- Continuous
- Single Agent

Actions

- Move Forward
- Turn Left / Right
- Collect Samples
- Capture Images
- Analyze Samples
- Avoid Obstacles
- Send Data to Earth

Performance Measures

- Accurate sample collection
- Safe navigation

- Efficient battery usage
- Successful data transmission
- Mission completion

Suitable Agent

- **Goal-Based Agent**

3. 8-Queens Problem

Solution

The 8-Queens Problem is solved using the **Backtracking Search** algorithm. The algorithm places one queen at a time on the chessboard while checking whether the position is safe. If the current position does not conflict with any previously placed queen, the queen is placed and the algorithm proceeds to the next column. If a conflict occurs, the algorithm removes the previous queen and tries another position. This process continues until all eight queens are placed safely without attacking each other.

Step-by-Step Solution

1. Start with an empty 8×8 chessboard.
2. Place the first queen in the first column.
3. Check whether the next queen can be placed safely.
4. If the position is safe, place the queen.
5. If not safe, try the next row.

6. If no safe position is available, backtrack to the previous queen.
7. Continue until all eight queens are placed safely.
8. Display the final board configuration.

Search Strategy: Backtracking (Depth-First Search)

4. OLA Cab Booking – Intelligent Agent

Solution

The OLA Cab Booking system uses a **Goal-Based Agent** to help customers travel from one location to another. The customer enters the pickup location, destination, and preferred cab type. The system checks the availability of nearby drivers, estimates the fare and arrival time, and assigns the most suitable cab. After confirmation, the customer starts the journey and reaches the destination safely.

Step-by-Step Solution

1. Open the OLA Cab Booking application.
2. Enter the pickup location.
3. Enter the destination.
4. Select the preferred cab type.
5. Check the availability of nearby drivers.
6. Calculate the estimated fare and arrival time.
7. Assign the nearest available driver.

8. Confirm the booking.
9. Complete the trip successfully.

Agent Used: Goal-Based Agent

5. Uniform Cost Search (UCS)

Solution

Uniform Cost Search (UCS) is used to find the **least-cost path** between the source node and the destination node in a weighted graph. The algorithm starts from the source node with a cost of zero and stores it in a priority queue. It always selects the node with the **lowest cumulative cost** for expansion. The neighboring nodes are explored, and their costs are updated. This process continues until the destination node is reached. Since UCS always expands the least-cost path first, it guarantees the optimal solution.

Step-by-Step Solution

1. Start from the source node **S**.
2. Set the initial path cost to **0**.
3. Insert the source node into the priority queue.
4. Remove the node with the lowest cumulative cost.
5. If the node is the destination **G**, stop the search.
6. Otherwise, expand all neighboring nodes.
7. Calculate the cumulative cost for each neighboring node.

8. Insert the neighboring nodes into the priority queue.
9. Repeat the process until the destination is reached.
10. Display the least-cost path and total cost.

Search Strategy: Uniform Cost Search (UCS)